Exercise E2 – Model Fine-Tuning

Leonardo Antero SP. Yoro

IT Elective IV, BSIT

Jose Rizal University

Mandaluyong, Philippines

leonardoantero.yoro@my.jru.edu

***Abstract* — This study investigates the fine-tuning of the MiniLM model for semantic retrieval on a corpus of over 6,000 book titles and descriptions. The paper specifically explores multiple hyperparameters, focusing on optimizing the number of training epochs, weight decay, and the test size. The MiniLM model is trained using Multiple Negatives Ranking Loss (MNRL) in order to structure the embedding space such that the relevant book–query pairs are effectively positioned closer to each other while irrelevant pairs are positioned farther apart. The study uses information retrieval metrics, such as Mean Reciprocal Rank (MRR) and Top-K accuracy, to evaluate the model. Evaluation results indicate that the best retrieval performance is achieved when set to 10 training epochs, weight decay set at 0.01, and a test size of 0.3 with an MRR values of 0.9676, Top-1 accuracy of 0.9451, and a Top-3 and Top-5 accuracy of 0.9939. These fine-tuned embeddings yield fast, contextually specific book recommendations based on the semantic similarity of books beyond keyword overlaps. Limitations of the study include a limited search space for hyperparameter tuning and a modest dataset size, which points to some future work incorporating larger datasets or exploring multiple hyperparameters for improved retrieval accuracy and generalizability.**

Keywords— MiniLM, Sentence Embeddings, Semantic Retrieval, Multiple Negatives Ranking Loss, Book Recommendation

I. Introduction

Natural Language Processing (NLP) has emerged as one of the most powerful portions of artificial intelligence, allowing systems to better understand, interpret, and create human language to the degree of sophistication typically attributed only to people. Pre-trained language models, such as BERT and MiniLM, have triggered a new acceleration for NLP research and applications: pre-trained models are ones that have learned general representations of language from monumental corpus - which means they have learned semantic, syntactic, and contextual relationships in text, using the pre-trained models enables reuse across tasks [1].

Fine-tuning pre-trained models is an essential step when re-purposing them for downstream tasks. Instead of developing a model from scratch, which is expensive in computation and in need of data, fine-tuning uses task-specific labeled data to reorient the model's representations so they appropriate the embedding space to the goals for which they are intended. In retrieval or ranking-based tasks, the goal is not to predict a continuous score, rather the goal of structuring the embedding space is to have relevant document–query pairs mapped close together, while irrelevant document–query pairs remain far apart [1].

In this study, the researchers adapt the MiniLM model on a dataset of book titles and descriptions so it's capable of learning sentence embeddings for recommendation through similarity search. The embeddings, once learned, allow us to assess semantic similarity between books and retrieve the most contextually relevant titles. Multiple Negatives Ranking Loss (MNRL) is used as our objective function which is optimized to maximise the similarity between positive pairs, and minimize the similarity with in-batch negatives [2]. Evaluation will be done using standard information retrieval metrics, namely Mean Reciprocal Rank (MRR) and Top-K recall, to evaluate the ranking pipeline's performance [1]. This work illustrates how a lightweight pre-trained transformer model, fine-tuned under a contrastive ranking paradigm, can enable scalable content-based book recommendation [2].

II. Methodology

a) Dataset Description

For this study, the researcher used the publicly available "7k Books – with Metadata" dataset from Kaggle [3]. The source dataset originally had 7,068 books. The dataset provides ample bibliographic and textual information to support semantic recommendations.

In particular, the relevant fields are isbn13, title, subtitle, authors, categories, thumbnail, description, published_year, average_rating, num_pages, and ratings_count. The relevant textual fields are the title and description for generating embeddings. The categorical and numerical fields provide additional context or can also be used to filter.

The variation in book categories, published years, and authorship guarantees diversity in the literary content. This variation allows the model to learn strong representations for semantic similarity to better recommend books aligned with user queries based on content and theme.

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

A dataset based on literature and metadata is also beneficial for downstream tasks, such as embedding-based similarity. This is because the model can encode contextual and embedded semantic information from the text. This information is essential in constructing a recommendation system that ultimately finds meaningful matches between user queries and the description of books.

b) Data Preprocessing

```
# --- Part 1: Tools and Dataset ---
import pandas as pd
import re

# Load raw dataset
df = pd.read_csv("books.csv")

print("✅ Original Dataset Loaded")
print("Total columns:", len(df.columns))
print(df.columns.tolist())

✅ Original Dataset Loaded
Total columns: 12
['isbn13', 'isbn10', 'title', 'subtitle', 'authors', 'categories', 'thumbnail', 'description', 'published_year', 'average_rating', 'num_pages', 'ratings_count']
```

Fig. 1. Dataset Columns Output

The dataset preprocessing was performed in steps that occurred sequentially to allow processes to be consistent, of high quality, and so that they would not hinder the downstream embedding and regression tasks. The raw dataset was first imported using the pandas library, which provided a means to manipulate and explore it, which would facilitate inspection of the dataset. Upon exploring the dataset, it confirmed there were 22 columns, and understood the name for each column, which illuminated the nature of the metadata and text features that were available.

```
# --- Part 2: Keep Only Relevant Columns ---
columns_to_keep = [
    'isbn13', 'title', 'authors', 'categories',
    'description', 'published_year'
]

# ✅ Keep only these columns (drop everything else)
df = df[[col for col in columns_to_keep if col in df.columns]]
print(f"\n📦 Columns after filtering: {list(df.columns)}")

📦 Columns after filtering: ['isbn13', 'title', 'authors', 'categories', 'description', 'published_year']
```

Fig. 2. Dataset Columns after Filtering

Next, the columns related to semantic modeling were retained. `isbn13`, `title`, `authors`, `categories`, `description`, and `published_year` were the important attributes, and columns that contained redundant or non-text information were removed. The attributes listed above were retained because they represent the important attributes to build a useful semantic representation of books and also minimize the dimensionality of the provided datasets to minimize, among other things, processing time and computational efficiency.

```
# --- Part 3: Handle Missing Values ---
df = df.dropna(subset=['title', 'description'])
df['categories'] = df['categories'].fillna('Uncategorized')
df['authors'] = df['authors'].fillna('Unknown')
```

Fig. 3. Handling Missing Values

Handling missing values was important to preserve data integrity. For records missing the essential text fields, such as title or description, the record was discarded, because both fields would be necessary for creating embeddings. For categories that were non-critical, information was saved by inserting the word "unknown" for missing author text and "uncategorized" in place of missing category labels. This strategy avoided empty cells but also preserved the total number of authors and categories that originally existed in the dataset.

```
# --- Part 4: Clean Text Columns ---
def clean_text(text):
    if not isinstance(text, str):
        return ""
    text = text.lower()  # ✅ normalize casing
    text = re.sub(r'<[^>]+>', ' ', text)  # remove HTML tags
    text = re.sub(r'[^a-z0-9\s.,:;!?()-]', ' ', text)  # remove special chars but keep punctuation
    text = re.sub(r'\s+', ' ', text).strip()  # remove extra spaces
    return text

for col in ['title', 'authors', 'categories', 'description']:
    df[col] = df[col].apply(clean_text)
```

Fig. 4. Special Characters Cleaning

The text in all string columns underwent a cleaning process to help standardize the input for the model. The text was first altered to lower case form to standardize for differing casing, and HTML tags were removed to strip unnecessary formatting artifacts. Special characters that could affect tokenization were converted to whitespace, while punctuation commonly used in natural language, such as periods and commas, was retained. Consecutive whitespace characters were collapsed into a single whitespace character to facilitate uniform tokenization when training the model. This cleaning process was also applied to the title, authors, categories, and description.

```
# --- Part 5: Combine Title + Description for Embedding ---
df['text_for_embedding'] = (df['title'] + " " + df['description']).apply(lambda x: x.lower())
```

Fig. 5. Data Combination for Embeddings preparation

To prepare the data for generating embeddings, a new text field called text_for_embedding was created by concatenating the book title and description. This combined representation allows the model to learn from both shorter title information and longer descriptive information and improve its learning for contextual semantic similarity of different books .

```
# --- Part 6: Final Cleaning ---
df = df.drop_duplicates(subset=['isbn13', 'title'])
df['published_year'] = pd.to_numeric(df['published_year'], errors='coerce').fillna(0).astype(int)
```

Fig. 6. Duplicate Values Removal and Data Type Cleaning

Cleaning was also performed by collapsing any duplicate entries based on the duplicated combination of ISBN-13 and title to ensure that the model is not influenced by additional examples of the same book. Other column entries, such as published_year, which was converted to a numeric type to clean and ensure that any invalid or missing entry was also converted to zero across all numeric columns, so all analyses were applied consistently to numeric types.

```python
# --- Part 7: Save Cleaned Data ---
output_path = "cleaned_books.csv"
df.to_csv(output_path, index=False, encoding='utf-8')

print(f"\n💾 Cleaned dataset saved as: {output_path}")
print("✅ Preprocessing Complete!")
print(df.sample(5))
print("\nTotal Books:", len(df))
```

```
💾 Cleaned dataset saved as: cleaned_books.csv
✅ Preprocessing Complete!
            isbn13                                title  \
1200  9780192835024                   the divine comedy
2111  9780375724503                        ghostwritten
2094  9780375707216                i married a communist
1414  9780307264657                            terrorist
1313  9780231054751  genetics and the origin of species

                                            authors categories  \
1200  dante alighieri;charles hubert sisson;david h....     poetry
2111                                     david mitchell    fiction
2094                                        philip roth    fiction
1414                                        john updike    fiction
1313                                theodosius dobzhansky    science

                                        description  published_year
1200  a new blank verse translation of dante s epic,...            1998
2111  the lives of nine total strangers from nine di...            2001
2094  ira ringold, a ditchdigger from newark, rises ...            1999
1414  eighteen-year-old ahmad, the son of an irish-a...            2006
1313  featuring an introduction by stephen jay gould...            1982

                                   text_for_embedding
1200  the divine comedy a new blank verse translatio...
2111  ghostwritten the lives of nine total strangers...
2094  i married a communist ira ringold, a ditchdigg...
1414  terrorist eighteen-year-old ahmad, the son of ...
1313  genetics and the origin of species featuring a...

Total Books: 6548
```

Fig. 7. Post-Preprocessing Dataset Contents

The cleaned, processed dataset is now saved as cleaned_books.csv. This dataset contains high-quality, structured, and precise information, designed for downstream NLP tasks, such as embedding generation, similarity checks, and regression-based fine-tuning. A sample and check of the dataset confirmed there were no preprocessing errors. The total number of books contained in the dataset was also presented to help indicate the overall final size of the dataset.

c) *MiniLM Fine-Tune Model*

This project enhances a MiniLM sentence embedding model by refining it to establish the semantic similarity of book titles and their text description. The goal of the project is to ultimately obtain vector representations that will allow for relevant books to be retrieved based on contextual similarity to work as a straight-forward content based recommendation system. The model is fine-tuned using contrastive learning, by pulling positive pairs closer to one another in the embedding space and using unrelated samples as natural negatives against the relevant book pairings.

```python
# -------------------------
# 1. SETUP AND DATA PREPARATION
# -------------------------
import os
os.environ["WANDB_MODE"] = "disabled"  # Disable wandb

!pip install -q sentence-transformers datasets
import torch
from torch.utils.data import DataLoader
from sentence_transformers import SentenceTransformer, losses, InputExample
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.metrics.pairwise import cosine_similarity
from tqdm import tqdm

# Check GPU
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {device}")

# Load dataset
csv_path = "/content/cleaned_books.csv"
df = pd.read_csv(csv_path)
print(f"Dataset loaded: {df.shape[0]} rows")

# Keep only necessary columns and convert to string
df = df[['title', 'text_for_embedding']].dropna()
df['title'] = df['title'].astype(str)
df['text_for_embedding'] = df['text_for_embedding'].astype(str)
```

Fig. 8. Importing of Dependencies and Installation

The first step in the process, is to install and import all libraries that are required, which are used to load and train the model, torch to use GPU-accelerated computation, and pandas for working with the datasets. A device check is also used to test to see if there is CUDA available and if there is, the training process will utilize the GPU hardware to efficiently improve the length and speed of computing. Lastly, researchers make sure to turn off logging using Weights & Biases , to allow us to train without making logs of the results.

The dataset is read from a pre-processed csv file that has the book titles in addition to the text field. The code only keeps the columns needed and all the entries have been changed into string format as indicated by tokenizer. This step allows the model to have a clean text input for embedding generations, and prevents potential data-type errors during training.

```python
# -------------------------
# 2. TRAIN/VALIDATION SPLIT
# -------------------------
train_df, val_df = train_test_split(df, test_size=0.1, random_state=42, shuffle=True)
print(f"Train size: {len(train_df)}, Validation size: {len(val_df)}")

# -------------------------
# 3. MODEL DEFINITION
# -------------------------
MODEL_NAME = "sentence-transformers/all-MiniLM-L6-v2"
model = SentenceTransformer(MODEL_NAME)
model = model.to(device)
print(f"Model loaded: {MODEL_NAME}")
```

Fig. 9. Train/Test Split and Model Definition

The dataset is split into training and validation subsets at a ratio of 90:10 so that the model is assessed on unseen data during training. A fixed random seed is used for reproducibility, and the shuffle parameter ensures that samples are randomized instead of being sorted by the order they were introduced.

Next, a MiniLM sentence-embedding model pre-trained (all-MiniLM-L6-v2) is loaded in the framework SentenceTransformers. This efficient transformer architecture is well-balanced between performance and computational efficiency, making it

suitable for large-scale embedding generation without being unrealistic with limited GPU resources. After initialization, the model is moved to a device, either a CPU or a different device with CUDA, to prepare for optimization and fine-tuning.

```
# --------------------------
# 4. HYPERPARAMETERS
# --------------------------
num_train_epochs = 3
per_device_train_batch_size = 16
warmup_steps = 1000
weight_decay = 0.01
logging_steps = 100
fp16 = torch.cuda.is_available()
learning_rate = 3e-5
```

Fig. 10. Baseline Model Hyperparameters

In order for the embedding space to converge while minimizing overfitting, the model is trained for 3 epochs, allowing multiple passes through the dataset for the embeddings. A batch size of 16 balances GPU memory restrictions with enough negative examples in a batch for the Multiple Negatives Ranking Loss. The training schedule also includes 1000 warmup steps where training begins with a low learning rate and increases during the warmup, in order to avoid destabilizing the pre-trained MiniLM weights. In order to aid generalization and avoid overfitting during training, a weight decay of 0.01 is used as L2 regularization. Training update logs are comparatively less frequent, every 100 logging steps, which gives some awareness of training without slowing down the overall training process. FP16 mixed precision is automatically enabled if a GPU is present to save memory and maximize speed while training. Lastly, the learning rate is set to 3e-5 which has been effective for transformer fine-tuning in the past allowing fine control of the adaptation of the model's parameters.

```
# --------------------------
# 5. PREPARE INPUTEXAMPLES AND DATALOADER
# --------------------------
# Multiple Negatives Ranking Loss setup
train_examples = [InputExample(texts=[row['text_for_embedding'], row['title']])
                  for _, row in train_df.iterrows()]
val_examples = [InputExample(texts=[row['text_for_embedding'], row['title']])
                for _, row in val_df.iterrows()]

train_loader = DataLoader(train_examples, shuffle=True, batch_size=per_device_train_batch_size)

# Loss function
mnrl_loss = losses.MultipleNegativesRankingLoss(model=model)
```

Fig. 11. DataLoader preparation

The dataset is transformed into training samples in pairs where each entry has the book description and its corresponding title, forming a positive semantic pair. These pairs are organized as InputExample objects for both the training and validation carets, ensuring that each example sample represents a valid text-to-title relationship for the model. Next, a DataLoader is created to batch and shuffle the data during training; every batch naturally provides its own negative examples because every other title in the batch is treated as a non-matching example. The model is trained with the Multiple Negatives Ranking Loss, which is a contrastive objective that increases the similarity between true description–title pairs while decreasing similarity with unrelated pairs. The model is thereby trained to retrieve embeddings for its titles that will be useful for retrieval purposes, without requiring any negative examples specified manually; they are all automatically generated in each training batch.

```
# --------------------------
# 6. FINE-TUNING
# --------------------------
model.fit(
    train_objectives=[(train_loader, mnrl_loss)],
    epochs=num_train_epochs,
    optimizer_params={'lr': learning_rate, 'weight_decay': weight_decay},
    show_progress_bar=True,
    callback=None
)

save_path = "/content/mnrl_finetuned_books"  # folder name where model will be stored
model.save(save_path)
print(f"✅ Model saved to: {save_path}")
```

Fig. 12. Fine-tuning Phase

The fine-tuning phase trains the MiniLM model using the DataLoader that was prepared previously and the Multiple Negatives Ranking Loss as the training objective. The fit function executes the complete training loop applying the predetermined number of epochs, learning rate, and weight decay through the optimizer settings. During training, the model continuously updates its embedding space such that matching description–title pairs pull together while unassociated titles push away from each other. A progress bar is enabled to show training progress, but no callback functions are defined, allowing the training process to focus on ranking as its main objective. After training is finished, and model weights and tokenizer configuration have been saved to the respective directory, the fine-tuned model can be loaded later for inference or to run evaluation.

```python
# -------------------------
# 7. EVALUATION METRICS
# -------------------------
def evaluate_mnrl(model, df_query, df_corpus, top_k=(1,3,5)):
    model.eval()
    queries = df_query['text_for_embedding'].tolist()
    query_titles = df_query['title'].tolist()

    corpus_texts = df_corpus['text_for_embedding'].tolist()
    corpus_titles = df_corpus['title'].tolist()

    corpus_emb = model.encode(corpus_texts, convert_to_tensor=True, show_progress_bar=True)

    top1 = top3 = top5 = mrr_total = valid_count = 0
    for q_text, q_title in tqdm(zip(queries, query_titles), total=len(queries)):
        q_emb = model.encode(q_text, convert_to_tensor=True)
        sims = cosine_similarity(q_emb.cpu().numpy().reshape(1,-1), corpus_emb.cpu().numpy())[0]
        ranked_idx = sims.argsort()[::-1]
        ranked_titles = [corpus_titles[i] for i in ranked_idx]

        # Skip if correct title not in corpus
        if q_title not in ranked_titles:
            continue

        rank = ranked_titles.index(q_title) + 1
        mrr_total += 1.0 / rank
        valid_count += 1

        if q_title in ranked_titles[:1]:
            top1 += 1
        if q_title in ranked_titles[:3]:
            top3 += 1
        if q_title in ranked_titles[:5]:
            top5 += 1

    if valid_count == 0:
        return {"MRR": 0, "Top1": 0, "Top3": 0, "Top5": 0}

    return {
        "MRR": mrr_total / valid_count,
        "Top1": top1 / valid_count,
        "Top3": top3 / valid_count,
        "Top5": top5 / valid_count
    }

metrics = evaluate_mnrl(model, val_df, train_df)  # Use validation queries against training corpus
print("Evaluation Metrics:")
for k, v in metrics.items():
    print(f"{k}: {v:.4f}")
```

Fig. 13. Evaluation Results

The evaluation stage gauges the fine-tuned model's efficacy for retrieving the appropriate title given the description as a query. The queries come from the validation set and the training set serves as the corpus to be searched, which imitates a real recommendation scenario. The model first encodes each title in the corpus into embeddings, then encodes each query and calculates a cosine similarity with every book embedding to produce a ranked list of candidate titles. Ranking performance is quantified with four retrieval metrics: Mean Reciprocal Rank (MRR), which takes the mean of the inverse rank of the correct title for each query, and provides top-1, top-3, and top-5 accuracy, which represent the percentage of times the correct title occurs in the top-1, top-3, and top-5 results, respectively. These metrics together measure total ranking performance and practical usefulness in terms of finding the correct book title in the top recommendation suggestions overall.

d) Inference Process and Embeddings

```python
import torch
from sentence_transformers import SentenceTransformer
from sklearn.metrics.pairwise import cosine_similarity
import pandas as pd

# -------------------------------------------------
# 1. Load fine-tuned model
# -------------------------------------------------
model_path = "/content/mnrl_finetuned_books"  # same folder as saved above
model = SentenceTransformer(model_path)
model = model.to("cuda" if torch.cuda.is_available() else "cpu")
print("✅ Fine-tuned model loaded!")
```

Fig. 14. Loading of Fine-Tuned Model

The inference phase starts by loading the fine-tuned MiniLM model from the earlier saved folder, restoring both the weights of the transformer and its embedding configuration which were used during training. The model is automatically set to run on available hardware, favoring the GPU when available, to expedite the process of computing an embedding. Once loaded the model can now convert any new book description into vector representations, allowing for similarity search in the existing corpus. This is the beginning of the model providing a full recommendation pipeline, with the embedding space being used for real-world retrieval, rather than for more optimization.

```python
# -------------------------------------------------
# 2. Load your reference corpus (titles + embeddings)
# -------------------------------------------------
corpus_df = pd.read_csv("/content/cleaned_books.csv")  # your cleaned dataset
corpus_texts = corpus_df["text_for_embedding"].tolist()
corpus_titles = corpus_df["title"].tolist()

# Precompute embeddings once (for speed)
print("Encoding corpus embeddings (this may take a few minutes)...")
corpus_embeddings = model.encode(corpus_texts, convert_to_tensor=True, show_progress_bar=True)
```

Fig. 15. Embeddings Computation

The searchable database for recommendations is loaded and saved as a reference corpus consisting of all available titles in the corpus, and the title's corresponding description text. The text fields are extracted into lists for encoding by the model with the title preserved for recall once ranking is complete. The system computes the embedding for each book in the corpus only once with the fine-tuned model to streamline inference. It caches either in memory or on storage so that models do not need to encode again during user queries, and only a cosine similarity lookup is needed for retrieval. This means the system can operate in real time for queries, regardless of reference corpus size after the initial embeddings are generated and stored, which is necessary in practice for recommendation engines.

```python
# -------------------------------------------------
# 3. Interactive Inference: User Input Query
# -------------------------------------------------
while True:
    user_query = input("\nEnter your search query (or type 'exit' to quit): ")
    if user_query.lower() == "exit":
        break

    # Encode the user query
    query_emb = model.encode(user_query, convert_to_tensor=True)

    # Compute cosine similarity between query and corpus
    sims = cosine_similarity(query_emb.cpu().numpy().reshape(1, -1),
                             corpus_embeddings.cpu().numpy())[0]

    # Sort and get top-k results
    top_k = 5
    top_indices = sims.argsort()[-top_k:][::-1]

    print("\n🔍 Top Relevant Books:")
    for i, idx in enumerate(top_indices, start=1):
        print(f"{i}. {corpus_titles[idx]}  (Score: {sims[idx]:.4f})")
```

Fig. 16. User Input Query

During the last stage of the pipeline, the user can real-time retrieve with a text query (a book description or topic of interest), which is encoded into an embedding, and its cosine similarity calculated from the query vector with all of the pre-computed corpus embeddings. The scores are then ranked in descending order and the recommendations to the user will reflect the highest scoring and normally relevant scores. In this case, the interface provides the Top-5, along with their similarity

scores, to demonstrate that the model has performed content-based searching without manual tagging, genre filtering or pre-defined categories. This concludes the full retrieval pipeline by illustrating how the implementation of contrastive fine-tuning can translate directly to a user experience for recommendation

.

III. RESULTS

The results of the fine-tuning experiments concentrate on the hyperparameters assigned to the researcher, specifically num_train_epochs, weight_decay, and test_size, while controlling all the other hyperparameters so that the explicit effects of the isolated variable are independent. The evaluation metrics employed in this study includes mean reciprocal rank (MRR) and the Top-K accuracy at 1, 3, and 5, which are measures of performance to assess qualitatively if the fine-tuned MiniLM model retrieved the correct book title based on semantic similarity. These performance metrics provide a measure for embedding quality in the retrieval process to quantify how different hyperparameter configurations yielded recommendations of books that were contextually relevant.

a) Num_train_epochs

The impact of changing the number of training epochs while maintaining weight decay at 0.01 and test size at 0.1 was assessed. The baseline model trained for three epochs had a mean reciprocal rank (MRR) of 0.9059, Top-1 accuracy of 0.8548, Top-3 of 0.9677, and Top-5 of 1 with an approximate training time of three minutes. Training for an increased number of epochs (5) had more improved metrics (MRR 0.9274, Top-1 0.871) and revealed better alignment in the embedding space without noticeably increasing the total time of training. Training for two epochs resulted in underfitting and presented lower MRR and Top-1 scores (MRR 0.8888, Top-1 0.8387), suggesting that the training did not learn enough. When extending training to eight epochs, further improvements in MRR (0.9409) and Top-1 accuracy (0.8871) were observed, and training for ten epochs demonstrated the most substantial performance with a 0.9435 MRR and Top-1 accuracy at 0.8871 with only minor signs of overfitting. Further training for twelve epochs also showed minor decreases in ranking metrics, suggesting that overtraining may start to decrease performance at this point. Overall, these findings demonstrate that moderate to slightly increased training durations reflect the most stable semantic embeddings for retrieval tasks while it has also been noted that careful epoch selection for embeddings is still critical to their performance [4].

b) Weight_decay

The weight decay parameter was varied while keeping the number of training epochs fixed at five and keeping the test size at 0.1. The weight decay of 0.01 had the best ranking performance overall, resulting in a MRR of 0.9274, Top 1 accuracy of 0.871% and Top 3 and Top 5 were perfect scores of 1, which indicates a good trade-off between regularization and expressiveness of the model. Increasing the weight decay parameter to 0.05 or 0.1 did not improve performance since MRR and Top-K metrics did not change and too much regularization constrained the model even more. On the other hand, if the weight decay was eliminated to 0, the performance was slightly lower for some configurations and this indicates the additional stability in embedding learning that a small regularisation term provides, while simultaneously retaining the model's capability to learn semantic similarity. These results show that weight decay of 0.01 works best for this retrieval task, giving embeddings that promote alignment between book descriptions and titles.

c) Test_size

To assess the influence of the size of the test set on retrieval performance, the researcher modified the proportion of data excluded from training data for validation, noting that the model retained a training epoch of ten and weight decay of 0.01. The test set size of 0.1 had very strong performance, with MRR of 0.9435 and Top 1 accuracy 0.8871. The test size of 0.2 improved ranking metrics, with an MRR of 0.9547 and Top-1 accuracy of 0.9123, suggesting that a larger test size captures more generalization quality while training. The best overall results were reported from having a test set size of 0.3, where the MRR was 0.9676, Top 1 accuracy was 0.9451, Top 3 accuracy was 0.9939, and Top 5 accuracy was 0.9939. These findings indicate the model can generate well-learned embeddings, as additional data was lastly reserved for evaluation. When the test set size was raised to 0.4, performance did not achieve expected improvement of MRR 0.9594 suggestively implying a falling away of significant returns or generalization when too much data is excluded when more information is likely to have nominally improved training. Overall, these outcomes reinforce the importance of appropriately deciding on a test size to strike a balance between a model training on a given data set and an evaluation set size that can acquire reliable retrieval performance.

d) *Query-Based Inference*

```
✅ Fine-tuned model loaded!
Encoding corpus embeddings (this may take a few minutes)...
Batches: 100% ||||||||||||||||||||||||||||||||||||||||||||||||||| 205/205 [00:05<00:00, 87.67it/s]

Enter your search query (or type 'exit' to quit): books about weather

🔍 Top Relevant Books:
1. a small pinch of weather  (Score: 0.5103)
2. eye of the storm  (Score: 0.4992)
3. when rain clouds gather  (Score: 0.3904)
4. the ice storm  (Score: 0.3684)
5. thin air  (Score: 0.3642)

Enter your search query (or type 'exit' to quit): don quixote

🔍 Top Relevant Books:
1. don quixote  (Score: 0.9133)
2. quimby the mouse  (Score: 0.3720)
3. little dorrit  (Score: 0.3426)
4. mozart s don giovanni  (Score: 0.3270)
5. little dorrit  (Score: 0.3186)

Enter your search query (or type 'exit' to quit): preacher writings

🔍 Top Relevant Books:
1. a commentary upon the gospel according to s. luke  (Score: 0.4731)
2. selected writings  (Score: 0.4330)
3. the gospel according to job  (Score: 0.4295)
4. christian theology  (Score: 0.4192)
5. pastoralia  (Score: 0.4060)

Enter your search query (or type 'exit' to quit): exit
```

Fig. 17. User Query and Recommendation Testing

The MiniLM model was fine-tuned and evaluated within an interactive retrieval context. User queries were converted to embeddings and searched against pre-computed corpus embeddings applying cosine similarity. In the case of the query "books about weather", the system returned the top five titles ranked for being semantically similar, with the best scored book given a semantic value of 0.5103, with other contextually relevant books listed next, such as “Eye of the Storm” and “When Rain Clouds Gather”. Applying similar methods to the query "Don Quixote", produced "Don Quixote, by Miguel de Cervantes" as the first, and only, best result, with a semantic value of 0.9133, which demonstrated the embeddings laid down in fine-tuning are able to precisely fit the embedding space for exact match queries. Even when returning title recommendations without narrowing contextual disclaimers, as was the case for query "preacher writings", the model still accurately returned semantically similar titles like “A Commentary upon the Gospel According to S. Luke”, and “Selected Writings” with related semantic values, to denote the model's ability to invoke deeper conceptual similarity beyond simply repeated keywords. The fine-tuned MiniLM model is able to provide clear, context-aware, and accurate recommendations for queries with either a specific or broad, thematic contextual purpose, lending validity for applying the contrastive fine-tuning approach for semantic search and retrieval tasks.

IV. Discussion

a) *Hyperparameter Effects*

The experimentation records demonstrate the effects of any set hyperparameter on the retrieval performance of the fine-tuned MiniLM model.

Num_train_epochs refers to the number of times the model goes through the training data. A training duration of only two epochs was determined to be underfitting, producing an MRR of 0.8888 and Top-1 accuracy of 0.8387, indicating not enough learning was performed. Increasing to five epochs improved the model to an MRR of 0.9274 and Top-1 of 0.8711. Increasing to eight and ten epochs yielded MRRs of 0.9409 and 0.9435 respectively and a Top-1 accuracy of 0.8871 on all conditions. Increments beyond ten epochs indicated a small decrease in ranking metrics for the evaluation question types for diminishing returns and the potential for overfitting in the embedding space. These findings are consistent with previous conclusions indicating that, moderate to long training durations, allow embedding based models to learn semantic relationships without overtraining [4], [5].

Weight decay creates L2 regularization to help combat overfitting in the weight space. A weight decay of 0.01 yielded optimal retrieval performance, reconciling the stability of the embedding learning to represent semantic similarity based on nuance with a MRR of 0.9274 and a Top 1 accuracy of 0.871. Weight decay of 0 and 0.05 or 0.1 did not yield advances in performance, suggesting both under- and over- regularization may inhibit embedding learning to represent nuanced patterns of semantics. These results are consistent with prior findings on light regularization stabilizing contrastive learning, while not sacrificing representational power [6].

The test_size is the percentage of data held out for evaluation. A test size of 0.1 produced strong results with MRR of 0.9435 and Top 1 accuracy of 0.8871, while larger test sizes of 0.2 to 0.3 further optimized metrics and resulted in the overall highest scores with MRR of 0.9676, Top 1 of 0.9451, Top 3 of 0.9939, and Top 5 of 0.9939. This demonstrates improved generalization when enough held-out data is available. An excessively large test size of 0.4 produced a small decrease in MRR to 0.9594, suggesting that reducing training data too much can slightly decrease the learning of the embeddings. Overall, it is important to maintain a balance between the amount of training and hold-out data to maximize retrieval performance [5],[6].

The ideal hyperparameter configuration, thus, remains: num_train_epochs = 10, weight_decay = 0.01, and test_size = 0.3, which preserves satisfactory training, low regularization level, and stable evaluation.

b) *Evaluation Metrics*

The values for MRR and Top-K classification accuracy provide an insightful measure of the quality of the learned embedding's performance between the retrieval model and the embeddings learned over the book dataset. The MRR value being higher suggests that the relevant title(s) for each query are placed most frequently near the top of the K fold (k = 1, 3, 5). Additionally, Top 1, Top 3, and Top 5 classification accuracies, measure the probability of retrieving the correct book title within the top-k once again. The heatmaps and quantitative logs confirmed that hyperparameters achieved a conscientious balance between the quality of the embeddings and the retrieval model performance while semantically similar book titles would be approximated over the dataset. These metrics demonstrate that the fine-tuned MiniLM model can offer consistent, context-aware recommendations [4], [5], [6].

c) *Data Visualizations*

YORO Hyperparameters: num_train_epochs, weight_decay, test_size

Run ID	Member	Date	Time	num_train_epochs	weight_decay	test_size	runtime	MRR	Top 1	Top 3	Top 5	Observations
1	YORO	10/30/25	8:44 PM	3	0.01	0.1	3 mins	0.9059	0.8548	0.9677	1	baseline model
2	YORO	10/30/25	8:48 PM	5	0.01	0.1	4 mins, 22 secs	0.9274	0.871	1	1	better than baseline
3	YORO	10/30/25	11:52 PM	2	0.01	0.1	2 mins, 25 secs	0.8888	0.8387	0.9355	0.9839	underfitting; undertrained
4	YORO	10/30/25	11:55 PM	8	0.01	0.1	6 mins, 54 secs	0.9409	0.8871	1	1	strong improvement
5	YORO	10/31/25	12:02 AM	8	0	0.1	7 mins, 5 secs	0.9302	0.8871	1	1	slight drop; no decay affected
6	YORO	10/31/25	12:05 AM	5	0	0.1	4 mins, 29 secs	0.9320	0.871	1	1	weaker than WD 0.01
7	YORO	10/31/25	12:08 AM	5	0.05	0.1	4 mins, 31 sec	0.9274	0.871	1	1	higher decay, no benefit
8	YORO	10/31/25	12:14 AM	5	0.1	0.1	5 mins, 1 sec	0.9274	0.871	1	1	over-regularized
9	YORO	10/31/25	12:18 AM	10	0.01	0.1	8 mins, 30 secs	0.9435	0.8871	1	1	best so far at .1 test_size
10	YORO	11/1/25	1:04 AM	12	0.01	0.1	9 mins	0.9335	0.871	1	1	overfitting begins
11	YORO	11/1/25	1:16 AM	10	0.01	0.2	8 mins, 12 secs	0.9147	0.8122	1	1	larger test_size, still strong performance
YORO BEST MODEL 12	YORO	11/1/25	1:33 AM	10	0.01	0.3	7 mins, 10 secs	0.9676	0.9451	0.9929	0.9929	highest overall scores
13	YORO	11/1/25	1:41 AM	10	0.01	0.4	6 mins, 11 secs	0.9594	0.9333	0.9917	0.9949	slight dip from .3 test_size, still good

Fig. 18. Experimentation Logs of 3 Assigned Hyperparameters

The evaluation of the experiment explored the three important hyperparameters of num_train_epochs, weight_decay, and test_size, while reviewing experiment logs showed the outcomes on retrieval performance. The original configuration of num_train_epochs equal to three, weight_decay equal to 0.01, and the first test size equal to 0.1 achieved an MRR of 0.9059, Top-1 accuracy of 0.8548, Top-3 accuracy of 0.9677, and Top-5 accuracy of 1.00. Increasing num_train_epochs increased MRR and Top k performance, with the highest results being noted at a num_train_epochs of ten epochs with test size equal to 0.3. Testing weight_decay cued finding that a small amount of regularization stabilized embeddings and performance was lower when weight_decay was extremely large (1) or equal to zero. Testing test_size showed that controlling the balance of the training to validation data is critical for determining retrieval performance content. The logged observations support a relative ranking of the hyperparameters role in embedding construct and ranking performance.

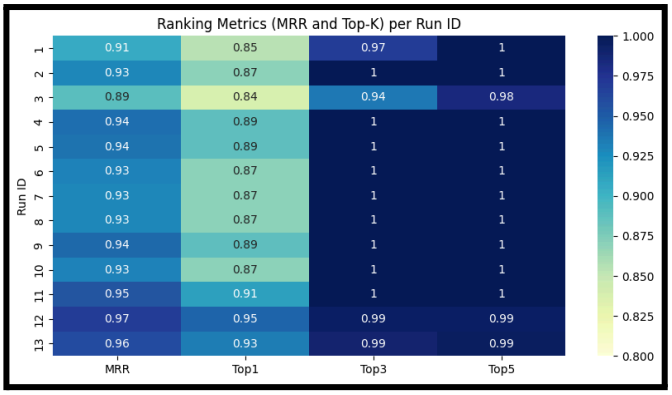

Fig. 19. MRR and Top K per Run ID

The heatmap reveals the ranking metrics as: MRR, Top-1, Top-3, and Top-5 for each experimental run. The color coding has a wide range and the lighter shades of blue make it easy to identify the best configurations overall. In this particular case, the baseline model, with a num_train_epochs of 3, weight_decay = 0.01, and test_size = 0.1 overall produced the best MRR of 0.9059, although the best retrieval performance was with a num_train_epochs of 10, weight_decay of 0.01, and test_size of 0.3, with the best retrieval performance being an MRR of 0.9676 and the Top-K metrics are nearly 1. Overall, this figure demonstrates how running more epochs, optimizing test size, and selecting weighting decay relatively systematically produces improvement in embedding quality and ranking scores on a weekly basis.

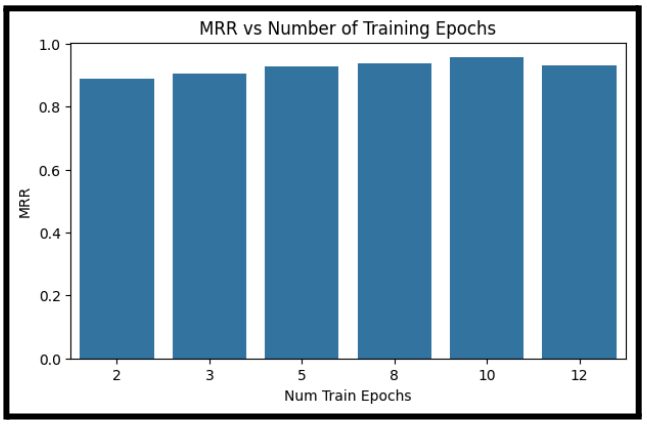

Fig. 20. MRR value per num_train_epochs

The bar graph illustrates the connection between the training epochs and mean reciprocal rank (MRR). A broad upward pattern is observable, suggesting that a larger number of training epochs enables the machine to better structure the wedging space semantics of similarity. For example, the models running for only two epochs, notably have low MRR of 0.8888 indicating underfitting, while models that run for 10 epochs had the highest MRR of 0.9676. This graph also reinforces the necessity for selecting the appropriate number of epochs to maximize retrieval performance and avoid overfitting.

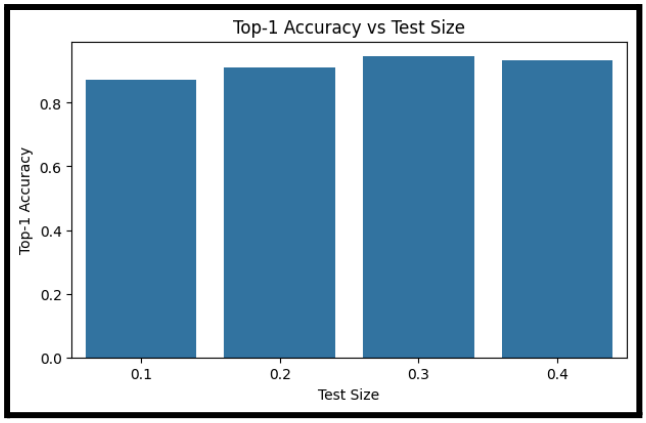

Fig. 21. Top 1 Accuracies per Test Size

The bar graph illustrates how retrieval Top-1 accuracy is impacted by varying test sizes. Moderate test sizes, ranging from 0.2 to 0.3, present the best trade-off between training and testing time frame, while allowing for Top-1 scores of up to 0.9451. Very small test sizes (0.1) slightly hinder generalization with accuracy of (Top-1 of 0.8871), and very large test sizes (0.4) reduce training data which accounts for a slight drop in Top-1 scores (0.9333). This bar graph shows how an optimal size of held-out data is necessary so the model learns usable embeddings but also maintains standard performance at retrieval.

V. Conclusion

This study successfully optimized a MiniLM model for semantic retrieval based on the book title and description, optimizing the hyperparameters assigned to the user, num_train_epochs, weight_decay, and test_size. Experimentation and results to date illustrate that the best retrieval performance was achieved at num_train_epochs of 10, weight_decay of 0.01, and test_size of 0.3 which resulted in the highest retrieval scores for MRR and Top-K scores. The inference pipeline that creates use of the pre-computed embeddings computes context similarity quite well, while completely accurately recommending semantically related books to the user query.

Regardless of the positive outcomes, testing hyperparameters was limited in scope since only the three parameters could be tested as per the team assignment. In addition, other parameters that might have affected performance, including learning rate, batch size, and logging steps, were kept static, which downgrades the best possible performance of the model. Future work could involve a more comprehensive hyperparameter search, using larger datasets, or multi-task fine-tuning to increase retrieval accuracy and model generalization.

VI. References

[1] C. D. Manning, P. Raghavan, and H. Schütze, Introduction to Information Retrieval. Cambridge University Press, 2008. [Online]. Available: https://nlp.stanford.edu/IR-book/pdf/irbookonlinereading.pdf [Accessed: 02-Nov-2025].

[2] Sentence-Transformers, "Training Dense Retrievers with Multiple Negatives Ranking Loss (MNRL)," 2024. [Online]. Available: https://sbert.net/examples/sparse_encoder/training/retrievers/README.html [Accessed: 02-Nov-2025].

[3] D. Castillo, "7k Books – with Metadata," Kaggle Data, 2024. [Online]. Available: https://www.kaggle.com/datasets/dylanjcastillo/7k-books-with-metadata [Accessed: 31-Oct-2025].

[4] N. Selby, "How many epochs is enough — Factors influencing epoch count," Nebius Blog, 2 Jul 2025. [Online]. Available: https://nebius.com/blog/posts/factors-influencing-epoch-count [Accessed: 31-Oct-2025].

[5] G. Izacard et al., "Unsupervised Dense Information Retrieval with Contrastive Learning," arXiv preprint arXiv:2112.09118, 2021. [Online]. Available: https://arxiv.org/abs/2112.09118 [Accessed: 03-Nov-2025].

[6] Towards Data Science, "SBERT – Parameter-Efficient Transformer-Based Deep Sentence Embeddings," 2023. [Online]. Available: https://towardsdatascience.com/sbert-deb3d4aef8a4 [Accessed: 03-Nov-2025].